NetworkAgent

AI-Powered Network Operations Agent

Setup, Launch & Connection Guide

CLI | Web Dashboard | AI Chat | Automation | SNMP | SSH | NETCONF | RESTCONF

92 Files | 24,000+ Lines | 8 Vendor Platforms | 17 Automation Actions

Version 1.0

. Table of Contents

1. Prerequisites & Requirements

System Requirements

- Python 3.10 or higher
- pip package manager
- Network access to target devices (SSH/SNMP/HTTPS)
- Windows, Linux, or macOS

Required Python Packages

All dependencies are listed in requirements.txt and will be installed automatically:

Package	Purpose
fastapi + uvicorn	Web dashboard & REST API
typer + rich	CLI interface
netmiko	SSH device connections
pysnmp	SNMP polling & discovery
ncclient	NETCONF device management
httpx	RESTCONF & HTTP requests
aiosqlite	Async SQLite database
cryptography (Fernet)	Credential encryption
pyyaml	Runbook YAML parsing
apscheduler	Scheduled job execution
anthropic	Claude AI agent (optional)

NOTE

The AI Chat interface requires an Anthropic API key. Set it in your .env file as ANTHROPIC_API_KEY=sk-ant-...

2. Installation

1 **Clone or copy the project**

Place the network_agent folder on your machine. The default location used in this guide is your Desktop.

2 **Open a terminal**

Navigate to the project directory:

```
cd network_agent
```

3 **Create a virtual environment (recommended)**

Isolate project dependencies from your system Python:

```
python -m venv venv

# Windows:
venv\Scripts\activate

# Linux/Mac:
source venv/bin/activate
```

4 **Install dependencies**

Install all required packages:

```
pip install -r requirements.txt
```

5 **Create your .env file**

Copy the example and fill in your settings:

```
copy .env.example .env

# Edit .env with your settings:
# ANTHROPIC_API_KEY=sk-ant-...  (for AI Chat)
# SECRET_KEY=your-random-secret  (for JWT auth)
# ENCRYPTION_KEY=               (auto-generated on init)
```

3. Initialize the Project

The init command sets up the database, creates necessary directories, and generates your encryption key for credential storage.

```
python run.py init
```

What init does:

- Creates the SQLite database with 11 tables (devices, credentials, configs, metrics, alerts, audit_log, runbook_executions, scheduled_jobs, etc.)
- Creates the data/runbooks/ directory and loads 5 sample runbooks
- Generates a Fernet encryption key for secure credential storage
- Sets up default alert rules (CPU > 90%, Memory > 85%, etc.)

Expected output:

```
NetworkAgent initialized successfully.
  Database: data/network_agent.db (11 tables)
  Runbooks: data/runbooks/ (5 loaded)
  Encryption key: generated
  Ready to launch.
```

IMPORTANT

Run init only once. Running it again is safe (it won't overwrite existing data), but it's not necessary after the first time.

4. Launch Interfaces

NetworkAgent provides three interfaces. You can run any of them independently.

Option A: Web Dashboard

```
python run.py web
```

- Opens a dark-themed single-page dashboard at http://localhost:8080
- Real-time WebSocket updates for metrics, alerts, and automation events
- Views: Dashboard, Devices, Config, Monitoring, Alerts, Discovery, Diagnostics, Automation, AI Chat
- JWT authentication (default credentials in .env)

Option B: CLI (Terminal)

```
python run.py cli
```

Full-featured terminal interface with Rich formatting. Use --help on any command:

```
python run.py cli --help
python run.py cli device --help
python run.py cli config --help
python run.py cli monitor --help
python run.py cli automation --help
```

Option C: AI Chat

```
python run.py chat
```

- Interactive Claude-powered agent with 17 network operation tools
- Natural language: "Show me all offline devices" or "Backup the config on switch-01"
- Safety classification: safe/write/destructive actions with confirmation gates
- Requires ANTHROPIC_API_KEY in .env

5. Adding Your First Device

Via CLI

```
python run.py cli device add \
  --hostname "core-switch-01" \
  --ip "192.168.1.1" \
  --type switch \
  --vendor cisco_ios \
  --model "Catalyst 9300" \
  --location "Server Room A"
```

Via Web Dashboard

1. **Navigate to Devices tab**
   Click 'Devices' in the sidebar
2. **Click '+ Add Device'**
   Fill in the form with device details
3. **Save**
   The device appears in the device table

Via AI Chat

```
You: "Add a Cisco switch at 192.168.1.1 called core-switch-01"
```

Supported Vendor Types

Vendor Key	Platform
cisco_ios	Cisco IOS / IOS-XE switches & routers
juniper_junos	Juniper JunOS devices
fortinet	FortiGate firewalls
paloalto	Palo Alto Networks firewalls
mikrotik	MikroTik RouterOS
aruba	Aruba / HPE switches & APs
pfsense	pfSense firewalls

6. Storing Credentials

Credentials are encrypted with Fernet (AES-128-CBC) and stored in the database. Each credential set gets a unique ID that you assign to devices.

Add SSH Credentials

```
python run.py cli credential add \
  --name "cisco-admin" \
  --username "admin" \
  --password "YourSecurePassword" \
  --enable-secret "EnablePassword"
```

Add SNMP Community

```
python run.py cli credential add \
  --name "snmp-readonly" \
  --community "public" \
  --snmp-version "2c"
```

Assign Credential to Device

```
python run.py cli device update core-switch-01 \
  --credential-id "cisco-admin"
```

SECURITY

Credentials are encrypted at rest using Fernet symmetric encryption. The encryption key is stored in your .env file. Keep .env secure and never commit it to version control.

7. Connecting to Devices

Once a device has credentials assigned, NetworkAgent can connect via SSH, SNMP, NETCONF, or RESTCONF depending on the vendor driver.

Test Connectivity

```
# Ping test
python run.py cli diag ping 192.168.1.1

# Port check (SSH)
python run.py cli diag port-check 192.168.1.1 22

# Port check (HTTPS/RESTCONF)
python run.py cli diag port-check 192.168.1.1 443

# Full health check
python run.py cli diag health core-switch-01
```

Run a Command on a Device

```
# Show running config
python run.py cli troubleshoot run-command \
  --device core-switch-01 \
  --command "show running-config"

# Show interfaces
python run.py cli troubleshoot run-command \
  --device core-switch-01 \
  --command "show ip interface brief"
```

Connection Protocols by Vendor

Vendor	SSH	SNMP	NETCONF	RESTCONF
cisco_ios	SSH (Netmiko)	SNMP v2c/v3	Yes	Yes
juniper_junos	SSH (Netmiko)	SNMP v2c/v3	Yes	Yes
fortinet	SSH (Netmiko)	SNMP v2c	No	Yes
paloalto	SSH (Netmiko)	SNMP v2c	No	Yes (XML API)

mikrotik	SSH (Netmiko)	SNMP v2c	No	Yes
aruba	SSH (Netmiko)	SNMP v2c/v3	No	Yes
pfsense	SSH (Netmiko)	SNMP v2c	No	Yes (xmlrpc)

8. Configuration Management

Backup Device Config

```
# Single device
python run.py cli config backup core-switch-01

# All devices
python run.py cli config backup-all
```

View & Compare Configs

```
# List backups for a device
python run.py cli config history core-switch-01

# Diff two backups
python run.py cli config diff <backup-id-1> <backup-id-2>
```

Deploy Configuration

```
# Deploy commands to a device
python run.py cli config deploy core-switch-01 \
  --commands "interface GigabitEthernet0/1" \
  --commands "description Uplink to Router" \
  --commands "no shutdown"

# Dry run (preview only)
python run.py cli config deploy core-switch-01 \
  --commands "..." --dry-run
```

Rollback

```
# Rollback to a previous config
python run.py cli config rollback core-switch-01 <backup-id>
```

SAFETY

Always use --dry-run first to preview changes before deploying to production devices. Config deployments are logged in the audit trail.

9. Monitoring & Alerts

Start Monitoring

```
# Poll a single device
python run.py cli monitor poll core-switch-01

# Poll all devices
python run.py cli monitor poll-all

# Start continuous monitoring (background)
python run.py cli monitor start --interval 60
```

View Metrics

```
# View latest metrics
python run.py cli monitor show core-switch-01

# Web dashboard: Monitoring tab with real-time charts
```

Alert Rules

Default alert rules are created on init:

Rule	Condition	Severity
CPU Critical	cpu_utilization > 90%	critical
Memory High	memory_utilization > 85%	warning
Interface Errors	error_count > 100	warning
Device Unreachable	ping_loss = 100%	critical
High Latency	latency > 200ms	warning

When monitoring detects a metric exceeding a threshold, alerts fire automatically. If automation runbooks are configured for that alert type, remediation begins autonomously.

10. Automation & Runbooks

The automation engine executes YAML-defined runbooks in response to alerts, schedules, webhooks, or manual triggers. Runbooks are stored in data/runbooks/.

View Runbooks

```
python run.py cli automation list
```

Included Sample Runbooks

Runbook	Trigger	Description
cpu-high-remediation	alert	Gathers CPU data, notifies, waits, re-checks, escalates
nightly-config-backup	schedule	Backs up all configs daily at 02:00 UTC
interface-error-remediation	alert	Clears counters, waits 5min, re-checks
device-unreachable-triage	alert	Multi-source ping + port checks, escalates
weekly-compliance-check	schedule	Weekly Saturday config audit

Run a Runbook Manually

```
python run.py cli automation run cpu-high-remediation

# With a specific device context
python run.py cli automation run cpu-high-remediation \
  --device core-switch-01
```

View Execution History

```
python run.py cli automation history
python run.py cli automation show-exec <execution-id>
```

View Scheduled Jobs

```
python run.py cli automation jobs
```

Audit Log

```
python run.py cli automation audit
```

Creating a Custom Runbook

Create a YAML file in data/runbooks/. Here's the structure:

```yaml
name: "my-custom-runbook"
version: "1.0"
description: "What this runbook does"
enabled: true
tags: ["custom"]

trigger:
  type: "alert"          # alert | schedule | webhook | manual
  alert_match:
    rule_id: "rule-cpu-critical"
    severity: ["critical"]

cooldown:
  per_device: 300        # Seconds before re-running for same device

limits:
  max_duration_seconds: 600
  max_concurrent: 1

actions:
  - name: "Step 1"
    action: "run_command"
    params:
      device_id: "{{device_id}}"
      command: "show processes cpu"
    output_var: "cpu_output"
    on_failure: "continue"
    timeout: 30

  - name: "Step 2"
    action: "notify"
    params:
      channel: "slack"
      message: "CPU info: {{cpu_output}}"
```

17 Available Action Types

Action	Description

run_command	Execute CLI command via SSH
poll_device	Poll device health metrics
backup_config	Backup single device config
backup_all	Backup all device configs
deploy_config	Push config commands to device
rollback_config	Rollback to previous config
check_interface_errors	Check interface error counters
check_device_health	Full device health check
ping_test	ICMP ping test
check_port	TCP port connectivity check
scan_subnet	SNMP subnet discovery scan
notify	Send alert notification
log	Write to audit log
wait	Pause execution (seconds)
condition	Conditional branching
escalate	Escalation chain trigger
http_request	External HTTP API call

11. AI Chat Agent

The AI Chat agent uses Claude with 17 network operation tools. It understands natural language and can perform any operation the CLI can.

Launch

```
python run.py chat

# Or via the web dashboard: AI Chat tab
```

Example Conversations

- "Show me all devices and their status"
- "Backup the config on core-switch-01"
- "What's the CPU utilization on all switches?"
- "Deploy 'no shutdown' on interface Gi0/1 of switch-02"
- "Run the cpu-high-remediation runbook on fw-01"
- "List all active alerts"
- "Scan subnet 10.0.1.0/24 for new devices"
- "Show me the last 5 config backups for router-01"

Safety Levels

Every AI tool has a safety classification:

Level	Examples	Behavior
Safe	Read-only operations: list devices, show metrics, view configs	No confirmation needed
Write	Modifications: deploy config, add device, update settings	Confirmation prompt
Destructive	Irreversible: delete device, rollback config, clear data	Explicit confirmation required

12. Supported Vendors & Platforms

Cisco IOS / IOS-XE (vendor key: cisco_ios)

Devices: Catalyst switches, ISR routers, ASR routers

Protocols: SSH, SNMP, NETCONF, RESTCONF

Juniper JunOS (vendor key: juniper_junos)

Devices: EX switches, SRX firewalls, MX routers

Protocols: SSH, SNMP, NETCONF, RESTCONF

Fortinet FortiOS (vendor key: fortinet)

Devices: FortiGate firewalls, FortiSwitch

Protocols: SSH, SNMP, REST API

Palo Alto PAN-OS (vendor key: paloalto)

Devices: PA-series firewalls, Panorama

Protocols: SSH, SNMP, XML API

MikroTik RouterOS (vendor key: mikrotik)

Devices: MikroTik routers, switches, APs

Protocols: SSH, SNMP, REST API

Aruba / HPE (vendor key: aruba)

Devices: Aruba switches, CX series, APs

Protocols: SSH, SNMP, REST API

pfSense (vendor key: pfsense)

Devices: pfSense/OPNsense firewalls

Protocols: SSH, SNMP, xmlrpc API

13. Project Structure

```
network_agent/
  run.py                    # Entry point
  config.yaml               # Global configuration
  requirements.txt          # Python dependencies
  .env                      # Secrets (API keys, encryption key)
  core/
    database.py             # SQLite with 11 tables
    models.py               # Pydantic models & enums
    credentials.py          # Fernet encryption manager
    connection_pool.py       # Async connection pooling
    exceptions.py           # 19 exception classes
  devices/
    base.py                 # BaseDevice ABC
    ssh_device.py           # Netmiko SSH wrapper
    snmp_device.py          # pysnmp SNMP wrapper
    netconf_device.py       # ncclient wrapper
    restconf_device.py      # httpx REST wrapper
    registry.py             # Device class registry
    vendors/               # 8 vendor implementations
  operations/
    config_manager.py       # Backup/deploy/diff/rollback
    monitor.py              # Health polling & metrics
    discovery.py            # SNMP/ping network scanning
    troubleshoot.py         # Diagnostics tools
  alerts/
    engine.py               # Alert evaluation & firing
    rules.py                # Default alert rules
    channels/               # Slack, email, webhook
  automation/
    engine.py               # Core orchestrator
    executor.py             # Runbook action runner
    actions.py              # 17 action type handlers
    runbook.py              # YAML parser & validator
    scheduler.py            # APScheduler cron jobs
    audit.py                # Audit trail logger
    webhook_ingress.py      # Inbound event API
  interfaces/
    cli/                    # Typer CLI commands
    web/                    # FastAPI dashboard + routes
```

```
  ai_agent/               # Claude tool-use agent
data/
  runbooks/               # YAML runbook definitions
  network_agent.db        # SQLite database
```

14. Troubleshooting

Common Issues

"ModuleNotFoundError" when running

Make sure you've activated your virtual environment and installed requirements:

```
pip install -r requirements.txt
```

"Connection refused" when connecting to a device

Verify the device IP is correct and SSH/SNMP is enabled on the device:

```
python run.py cli diag ping 192.168.1.1
python run.py cli diag port-check 192.168.1.1 22
```

"Authentication failed"

Check that the credential is properly assigned to the device and the username/password are correct:

```
python run.py cli credential list
python run.py cli device show core-switch-01
```

Web dashboard not loading

Check that port 8080 is not in use and the init has been run:

```
python run.py init
python run.py web --port 8081  # Use alternate port
```

AI Chat says "API key not found"

Set your Anthropic API key in the .env file:

```
ANTHROPIC_API_KEY=sk-ant-api03-...
```

Getting Help

- CLI help: python run.py cli --help
- Command help: python run.py cli <command> --help

- Check logs in the terminal output for detailed error messages
- Audit log: python run.py cli automation audit